Chapter 2

Mobile Application Models

Contents

Route and Navigator

Route and Navigator

In Flutter, screens and pages are called **routes**.

In Android, a route is equivalent to an **Activity**.

In iOS, a route is equivalent to a **ViewController**.

In Flutter, a **route** is just a **widget**.

Route and Navigator

When a user interacts with your app, they navigate between these routes.

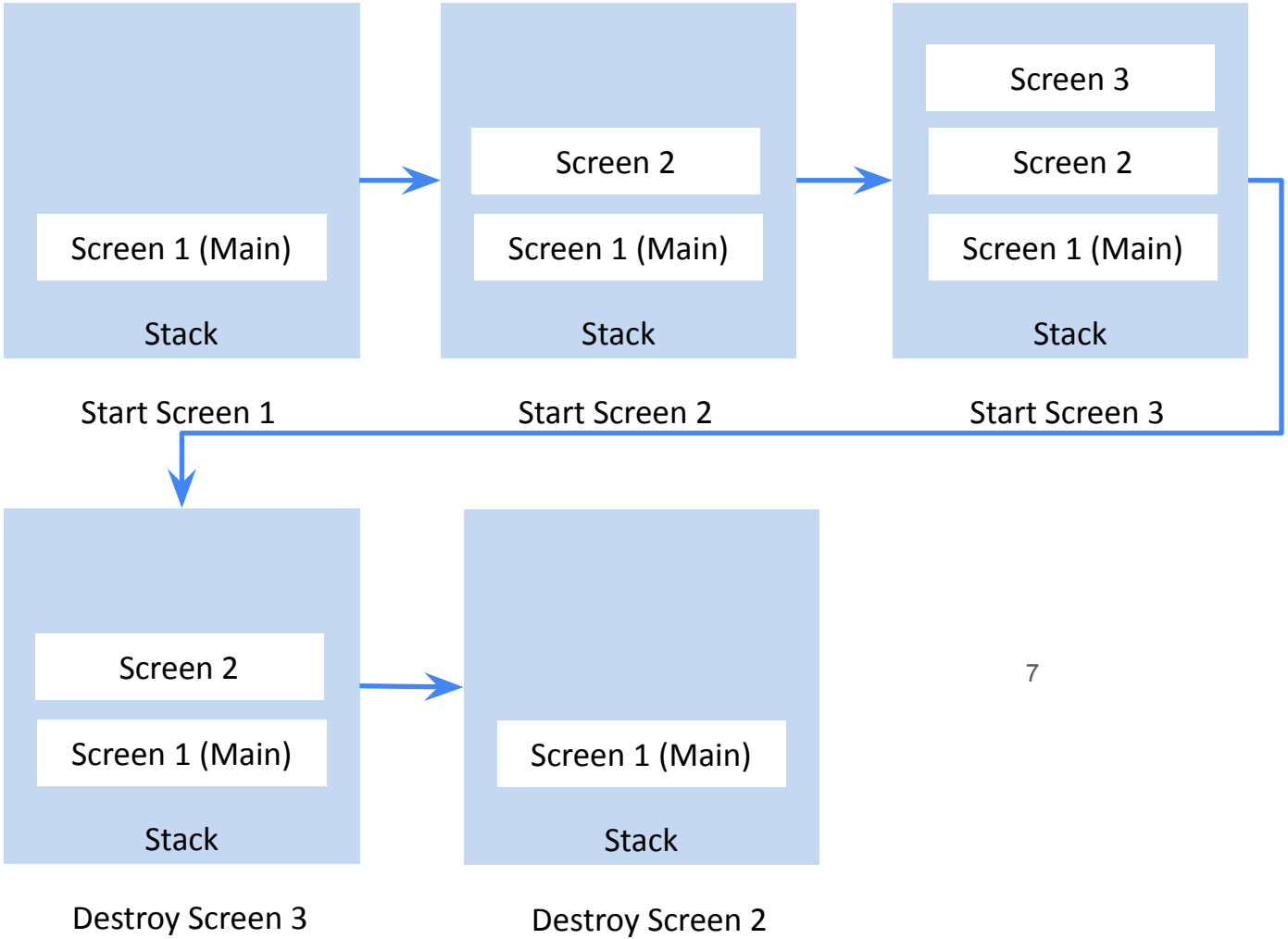
For advanced navigation and routing requirements, use a routing package such as `go_router`.

Route and Navigator

The Navigator widget displays screens as a **stack** using the correct transition animations for the target platform.

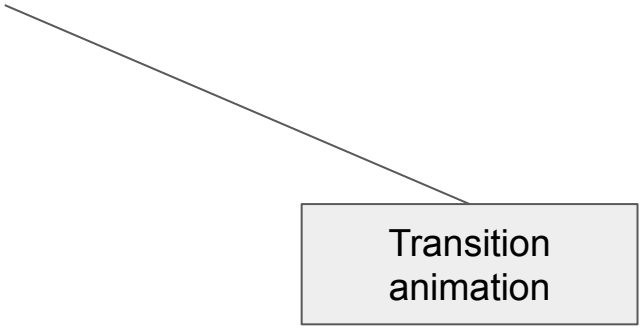
Navigator is suitable for small applications without complex deep linking.

To navigate to a new screen, access the Navigator through the route's `BuildContext` and call imperative methods such as `push()` or `pop()`:


Screen 1 (Main)
Stack
Start Screen 1
Screen 2
Screen 1 (Main)
Stack
Start Screen 2
Screen 3
Screen 2
Screen 1 (Main)
Stack
Start Screen 3
Screen 2
Screen 1 (Main)
Stack
Destroy Screen 3
Screen 1 (Main)
Stack
Destroy Screen 2

Route and Navigator

```
// Within the First Route
onPressed: () {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const SecondRoute()),
  );
}
```



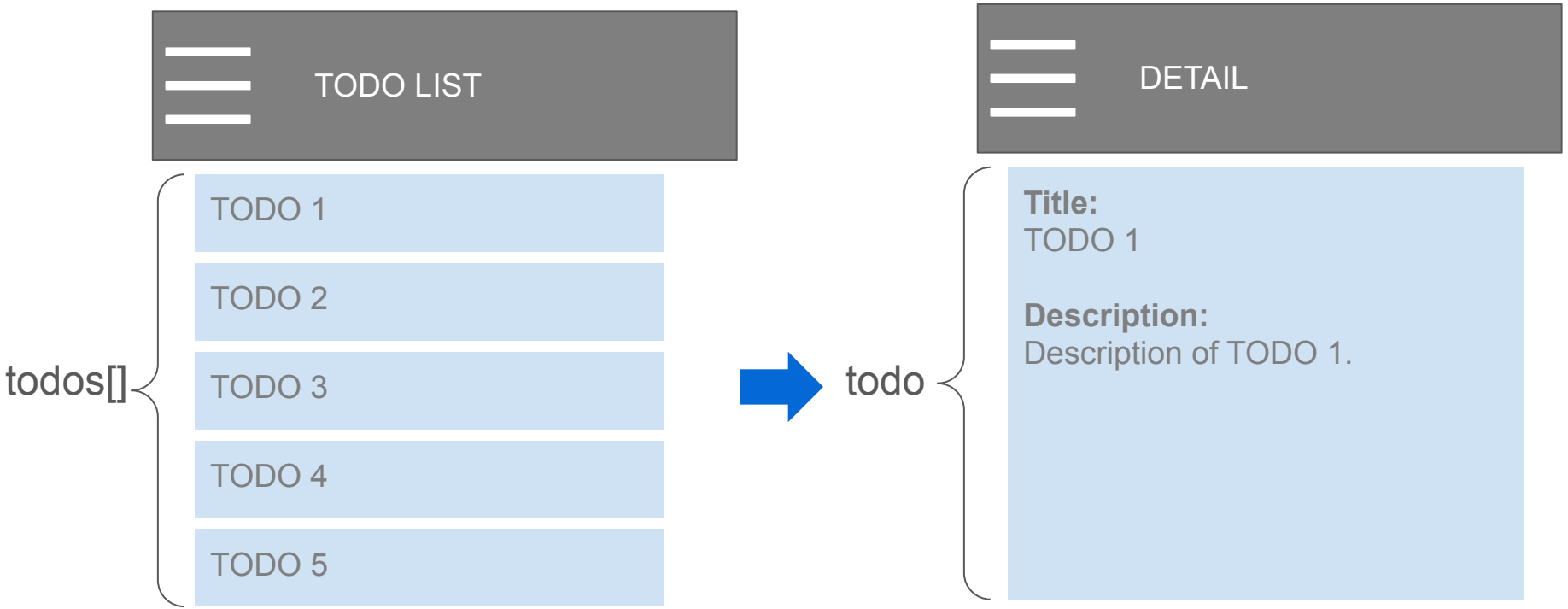
Route and Navigator

```
// Within the Second Route widget
onPressed: () {
  Navigator.pop(context);
}
```

Route and Navigator - Send Data

Often, you not only want to navigate to a new screen, but also pass data to the screen as well.

For example, you might want to pass information about the item that's been tapped.


TODO LIST
TODO 1
TODO 2
TODO 3
TODO 4
TODO 5
todos[]
DETAIL
Title:
TODO 1
Description:
Description of TODO 1.
todo

Route and Navigator - Send Data

```
class Todo {
  final String title;
  final String description;

  const Todo(this.title, this.description);
}
```

Route and Navigator - Send Data

```
class DetailScreen extends StatelessWidget {
  // In the constructor, require a Todo.
  const DetailScreen({super.key, required this.todo});

  // Declare a field that holds the Todo.
  final Todo todo;

  ...
```

Route and Navigator - Send Data

```
//In the Main Screen
Navigator.push(
          context,
          MaterialPageRoute(
            builder: (context) => DetailScreen(todo: todos[index]),
          ),
```

Refer to the complete code at https://docs.flutter.dev/cookbook/navigation/passing-data

Route and Navigator - Return Data

Often, you might want to pass data back from a second screen to the first screen after a user interaction, such as selecting an item or entering information.

```
//Within the Second Route
Navigator.pop(context, ‘Returned Data’);
```

```
//Within the Second Route
Navigator.pop(context, ‘Returned Data’);

// Within the First Route
onPressed: () {
  Navigator.push(
     context,
     MaterialPageRoute(builder: (context) => SecondScreen()),
     ).then((value) {
         // Handle the returned data here
          if (value != null) {
            setState(() { _dataFromSecondScreen = value; });
          }
       }
  );
}
```

Shared State Management

For more complex scenarios, consider using a state management solution like `Provider`.

This allows you to share data across multiple screens and update it from any part of your app.

Shared State Management

```
// Provider
class DataProvider extends ChangeNotifier {
   String _data = '';
   String get data => _data;

   void setData(String newData) {
      _data = newData;
      notifyListeners();
   }
}
```

Shared State Management

```
// First Screen
Consumer<DataProvider>(
   builder: (context, dataProvider, child) {
     return ElevatedButton(
       onPressed: () {
         Navigator.push(
           context,
           MaterialPageRoute(
             builder: (context) => SecondScreen(dataProvider:
                                   dataProvider),
           ),
         );
       },
      child: Text('Go to Second Screen'), );
   },
)
```

Shared State Management

```
// Second Screen
SecondScreen(this.dataProvider);

// ...

onPressed: () {
  dataProvider.setData('Returned data');
  Navigator.pop(context);
}
```

Deep Linking

Deep linking is the technique of linking directly to specific content within an app.

It allows users to open your app to a particular screen or feature, rather than just the home screen.

Flutter supports deep linking on iOS, Android, and the web.

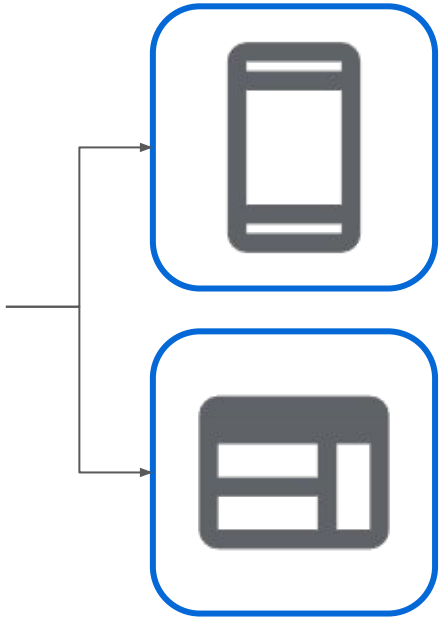
Opening a URL displays that screen in your app.

Deep Linking

You can use the same link for both web and app.

https://dashcast.net/library/album/1

Scheme | Domain | Path



Deep Linking

An **app link** is a type of deep link that uses `http` or `https` and is exclusive to Android devices. Find out more about it here: https://docs.flutter.dev/cookbook/navigation/set-up-app-links

Universal link is a deep link implementation for iOS. Find out more about it here: https://docs.flutter.dev/cookbook/navigation/set-up-universal-links

App Lifecycle

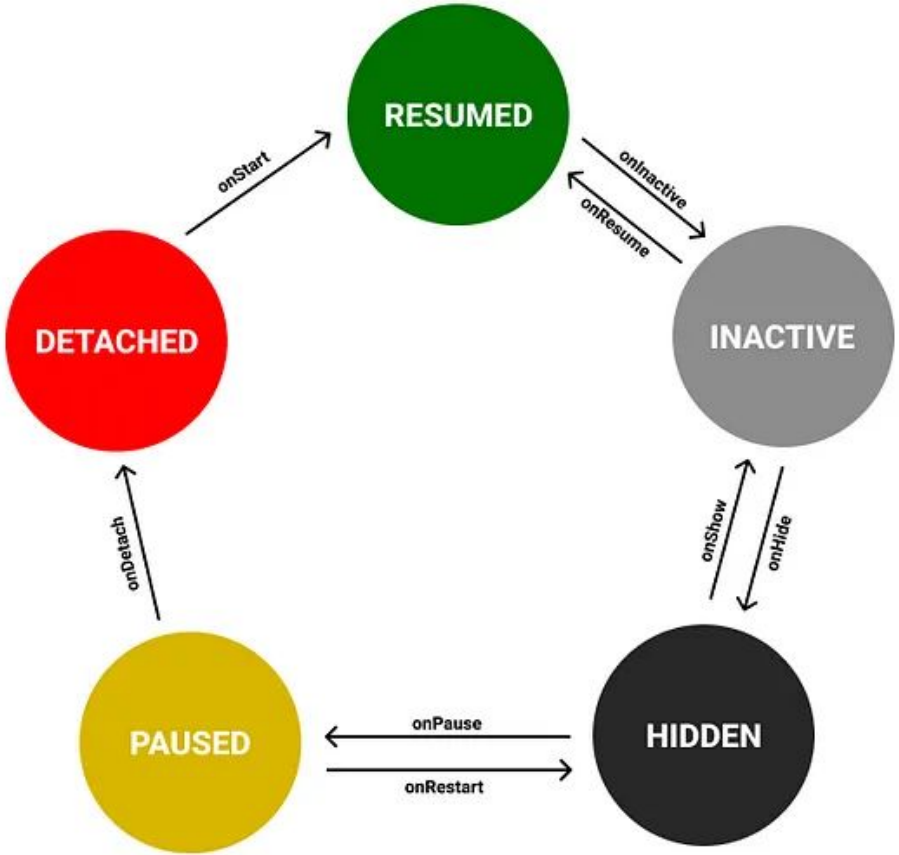
App Lifecycle

- The Flutter app lifecycle refers to the different states a Flutter app can be in throughout its runtime.

- Understanding these states and how the app transitions between them is essential for building efficient and responsive apps.

- By understanding the app lifecycle, you can create a more polished and user-friendly experience in your Flutter applications.

FLUTTER APPLICATION LIFE CYCLE

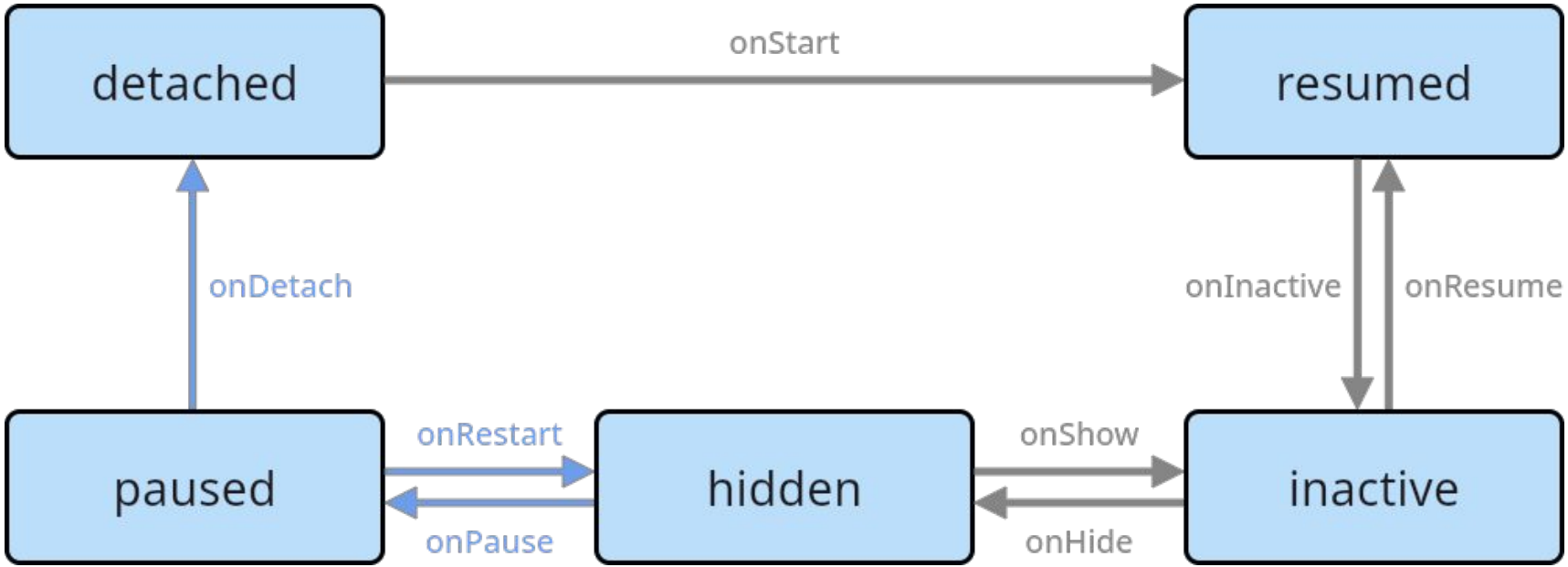


App Lifecycle

There are four primary states in the Flutter app lifecycle:

1. Resumed
2. Paused
3. Inactive
4. Detached

App Lifecycle



App Lifecycle

1. Resumed

This is the ideal state where the app is **visible** on the screen, actively **responding to user interactions**.

Here are some actions you typically take:

- Fetch and display data relevant to the user.
- Process user interactions (taps, swipes, form submissions).
- Play audio or video.
- Start animations.

Essentially, any functionality core to your app's active use happens here.

App Lifecycle

2. Paused

In this state, the app is **not visible** to the user and has **limited functionality**.

It's usually in the background. This is a good time for:

- Pausing tasks that consume resources (like animations or location updates).
- Saving any unsaved user data.
- Releasing resources that aren't immediately needed.

You want to **avoid intensive tasks** here as the app might be suspended entirely soon.

App Lifecycle

3. Inactive

This is a **brief transitional** state between **resumed** and **paused**, where the app might **lose focus** but still resides in memory.

You might **not have time for complex actions** here, but you could:

- Briefly pause ongoing tasks (like a network request) in case the app resumes quickly.
- Prepare for a potential state change (resumed or detached).

App Lifecycle

4. Detached

This state signifies the app is **no longer attached to any view** and **isn't actively running**.

It occurs before initialization and potentially after all views are detached (on Android and iOS).

This is a good time for:

- Cleaning up resources (closing databases, releasing memory).
- Persisting any critical data that needs to be saved even after the app is closed.

Avoid actions that require UI elements or user interaction.

App Lifecycle

Here are some key points to remember about the Flutter app lifecycle:

- You can monitor app lifecycle changes using the `AppLifecycleState` enum and the `AppLifecycleListener` class.

- Effectively handling lifecycle transitions allows you to optimize resource usage, perform actions when the app goes into the background (like pausing timers or saving data), and respond appropriately when it resumes.

```
@override
  void initState() {
    super.initState();
    _state = SchedulerBinding.instance.lifecycleState;
    _listener = AppLifecycleListener(
      onShow: () => _handleTransition('show'),
      onResume: () => _handleTransition('resume'),
      onHide: () => _handleTransition('hide'),
      onInactive: () => _handleTransition('inactive'),
      onPause: () => _handleTransition('pause'),
      onDetach: () => _handleTransition('detach'),
      onRestart: () => _handleTransition('restart'),
      // This fires for each state change. Callbacks above fire only for
      // specific state transitions.
      onStateChange: _handleStateChange,
    );
    if (_state != null) {
        _states.add(_state!.name);
    }
  }
```

Find Out More

Route and Navigator, https://docs.flutter.dev/ui/navigation

AppLifeCycleListener class,
https://api.flutter.dev/flutter/widgets/AppLifecycleListener-class.html